Assignment - 1

IPL Data Analysis

Objective:

This assignment analyzes IPL match data from 2008–2019 to identify patterns in team performance, match outcomes, player contributions, and venue influence. The goal is to extract meaningful insights that help understand key factors affecting match results in the Indian Premier League

Methodology:

In this project, IPL data from 2008–2019 was analyzed using Python. First, the datasets were loaded and explored to understand the structure of the data. Missing values and unnecessary columns were handled during the data cleaning stage.

After cleaning the data, different analysis techniques were applied to answer the business questions provided in the assignment. Various visualizations such as bar charts and line charts were created to understand patterns in team performance, venue distribution, toss decisions, and player contributions.

The analysis helped identify important insights about match results, team success, and player performance in the IPL.

Tools and Technologies Used:

• Python – Used as the main programming language for data analysis.

• Pandas – Used for loading, cleaning, and analyzing the dataset.

• NumPy – Used for basic numerical operations.

• Matplotlib – Used to create charts and visualizations.

• Seaborn – Used for better statistical visualizations.

• Jupyter Notebook – Used as the environment to run the Python code and perform the analysis.

In [22]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load datasets [cite: 20]
m = pd.read_csv(r'D:\College Stuffs\DV\matches.csv')
d = pd.read_csv(r'D:\College Stuffs\DV\deliveries.csv')
```

In [23]:
```
m.head()
```

Out[23]:

	id	season	city	date	team1	team2	toss_winner	toss_decision	resu
0	1	2017	Hyderabad	2017-04-05	Sunrisers Hyderabad	Royal Challengers Bangalore	Royal Challengers Bangalore	field	norma
1	2	2017	Pune	2017-04-06	Mumbai Indians	Rising Pune Supergiant	Rising Pune Supergiant	field	norma
2	3	2017	Rajkot	2017-04-07	Gujarat Lions	Kolkata Knight Riders	Kolkata Knight Riders	field	norma
3	4	2017	Indore	2017-04-08	Rising Pune Supergiant	Kings XI Punjab	Kings XI Punjab	field	norma
4	5	2017	Bangalore	2017-04-08	Royal Challengers Bangalore	Delhi Daredevils	Royal Challengers Bangalore	bat	norma

In [24]:
```
d.head()
```

Out[24]:

	match_id	inning	batting_team	bowling_team	over	ball	batsman	non_striker	bowle
0	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	1	DA Warner	S Dhawan	T Mill
1	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	2	DA Warner	S Dhawan	T Mill
2	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	3	DA Warner	S Dhawan	T Mill
3	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	4	DA Warner	S Dhawan	T Mill
4	1	1	Sunrisers Hyderabad	Royal Challengers Bangalore	1	5	DA Warner	S Dhawan	T Mill

5 rows × 21 columns

Dataset Description:

The dataset used in this assignment consists of two CSV files containing information about IPL matches and ball-by-ball deliveries from 2008 to 2019.

• matches.csv – Contains match-level information such as teams, venue, toss result, match outcome, and player of the match.

• deliveries.csv – Contains ball-by-ball information including batsman, bowler, runs scored, extras, dismissals, and innings details.

These two datasets allow both match-level analysis and player-level performance analysis, enabling deeper insights into team strategies and match outcomes.

In [25]:
```
m.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 756 entries, 0 to 755
Data columns (total 18 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   id               756 non-null    int64 
 1   season           756 non-null    int64 
 2   city             749 non-null    object
 3   date             756 non-null    object
 4   team1            756 non-null    object
 5   team2            756 non-null    object
 6   toss_winner      756 non-null    object
 7   toss_decision    756 non-null    object
 8   result           756 non-null    object
 9   dl_applied       756 non-null    int64 
 10  winner           752 non-null    object
 11  win_by_runs      756 non-null    int64 
 12  win_by_wickets   756 non-null    int64 
 13  player_of_match  752 non-null    object
 14  venue            756 non-null    object
 15  umpire1          754 non-null    object
 16  umpire2          754 non-null    object
 17  umpire3          119 non-null    object
dtypes: int64(5), object(13)
memory usage: 106.4+ KB
```

In [26]:
```
d.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 179078 entries, 0 to 179077
Data columns (total 21 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   match_id          179078 non-null  int64
 1   inning            179078 non-null  int64
 2   batting_team      179078 non-null  object
 3   bowling_team      179078 non-null  object
 4   over              179078 non-null  int64
 5   ball              179078 non-null  int64
 6   batsman           179078 non-null  object
 7   non_striker       179078 non-null  object
 8   bowler            179078 non-null  object
 9   is_super_over     179078 non-null  int64
 10  wide_runs         179078 non-null  int64
 11  bye_runs          179078 non-null  int64
 12  legbye_runs       179078 non-null  int64
 13  noball_runs       179078 non-null  int64
 14  penalty_runs      179078 non-null  int64
 15  batsman_runs      179078 non-null  int64
 16  extra_runs        179078 non-null  int64
 17  total_runs        179078 non-null  int64
 18  player_dismissed  8834 non-null    object
 19  dismissal_kind    8834 non-null    object
 20  fielder           6448 non-null    object
dtypes: int64(13), object(8)
memory usage: 28.7+ MB
```

In [27]:
```
m.isnull().sum()
```

Out[27]:
```
id                   0
season               0
city                 7
date                 0
team1                0
team2                0
toss_winner          0
toss_decision        0
result               0
dl_applied           0
winner               4
win_by_runs          0
win_by_wickets       0
player_of_match      4
venue                0
umpire1              2
umpire2              2
umpire3            637
dtype: int64
```

In [28]:
```
d.isnull().sum()
```

Out[28]:
```
match_id                 0
inning                   0
batting_team             0
bowling_team             0
over                     0
ball                     0
batsman                  0
non_striker              0
bowler                   0
is_super_over            0
wide_runs                0
bye_runs                 0
legbye_runs              0
noball_runs              0
penalty_runs             0
batsman_runs             0
extra_runs               0
total_runs               0
player_dismissed    170244
dismissal_kind      170244
fielder             172630
dtype: int64
```

In [29]:
```
m['date'] = pd.to_datetime(m['date'], format='mixed', dayfirst=True)
```

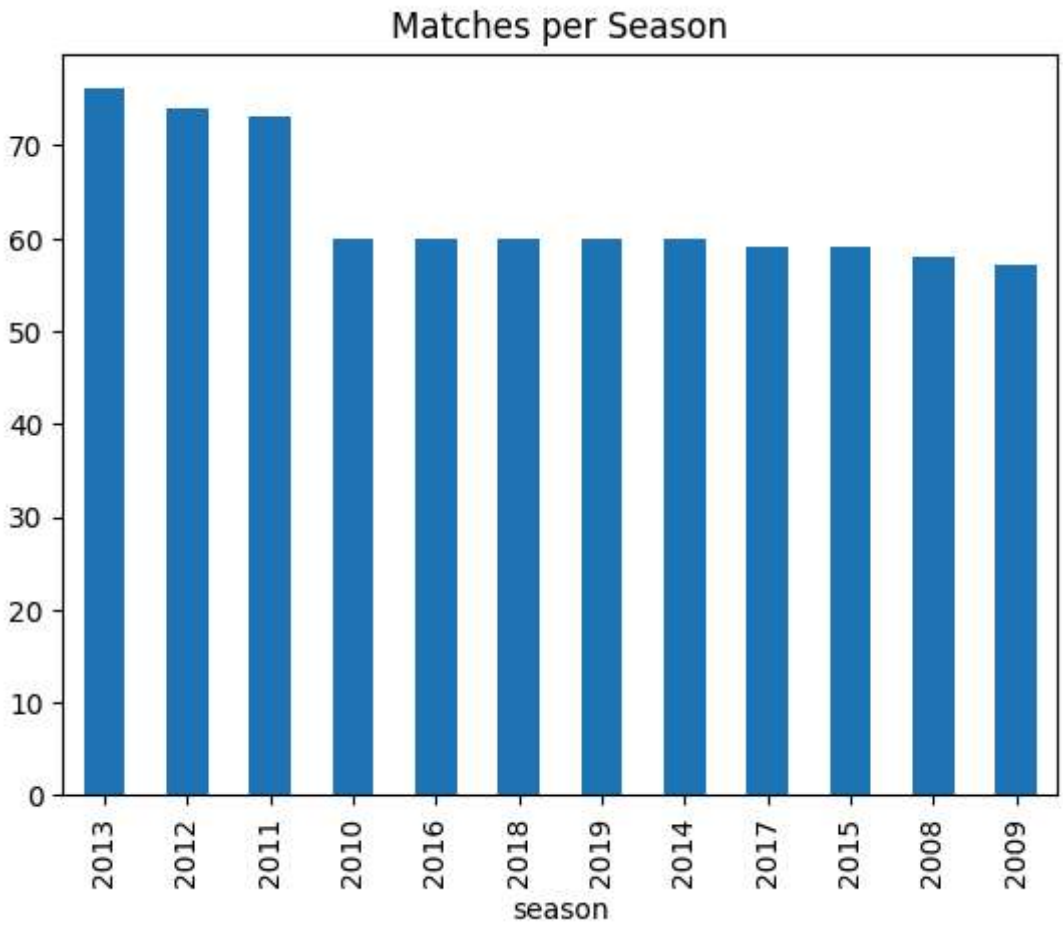
In [30]:
```
m['date'].head()
```

Out[30]:
```
0   2017-04-05
1   2017-04-06
2   2017-04-07
3   2017-04-08
4   2017-04-08
Name: date, dtype: datetime64[ns]
```

In [31]:
```
# Max and Min margins [cite: 65, 66, 67, 68]
print("Max runs win:", m.iloc[m['win_by_runs'].idxmax()]['winner'])
print("Max wickets win:", m.iloc[m['win_by_wickets'].idxmax()]['winner'])
print("Min runs win:", m[m['win_by_runs']>0]['win_by_runs'].min())
print("Min wickets win:", m[m['win_by_wickets']>0]['win_by_wickets'].min())
```

```
Max runs win: Mumbai Indians
Max wickets win: Kolkata Knight Riders
Min runs win: 1
Min wickets win: 1
```

In [32]:
```
# Counts per category [cite: 70, 71, 72]
m['city'].value_counts()
m['winner'].value_counts()
m['season'].value_counts().plot(kind='bar', title='Matches per Season')
plt.show()
```



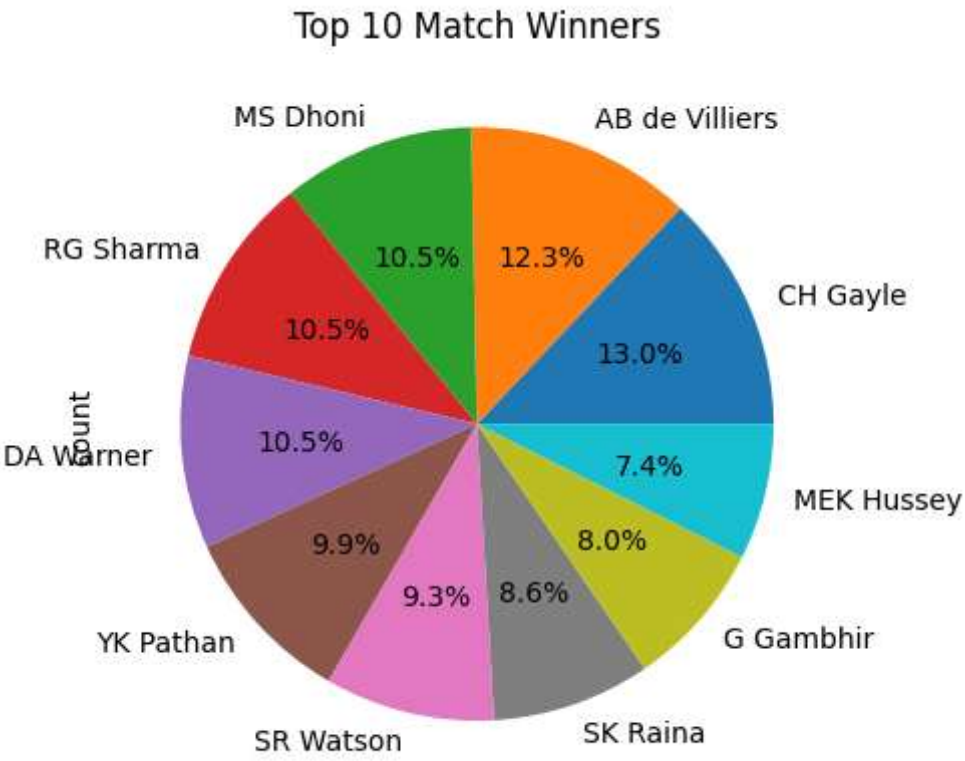
Matches per Season Insight:

• The number of matches peaked between 2011 and 2013, reaching nearly 76 matches per season.

• This increase was likely due to the expansion of the league with more franchise teams.

• Post-2013, the schedule stabilized to approximately 60 matches per year, indicating a consistent tournament format.

In [33]:
```
# Top 10 Players [cite: 73]
m['player_of_match'].value_counts().head(10).plot(kind='pie', autopct='%1.1f%%')
plt.title('Top 10 Match Winners')
plt.show()
```



Top 10 Match Winners Insight:

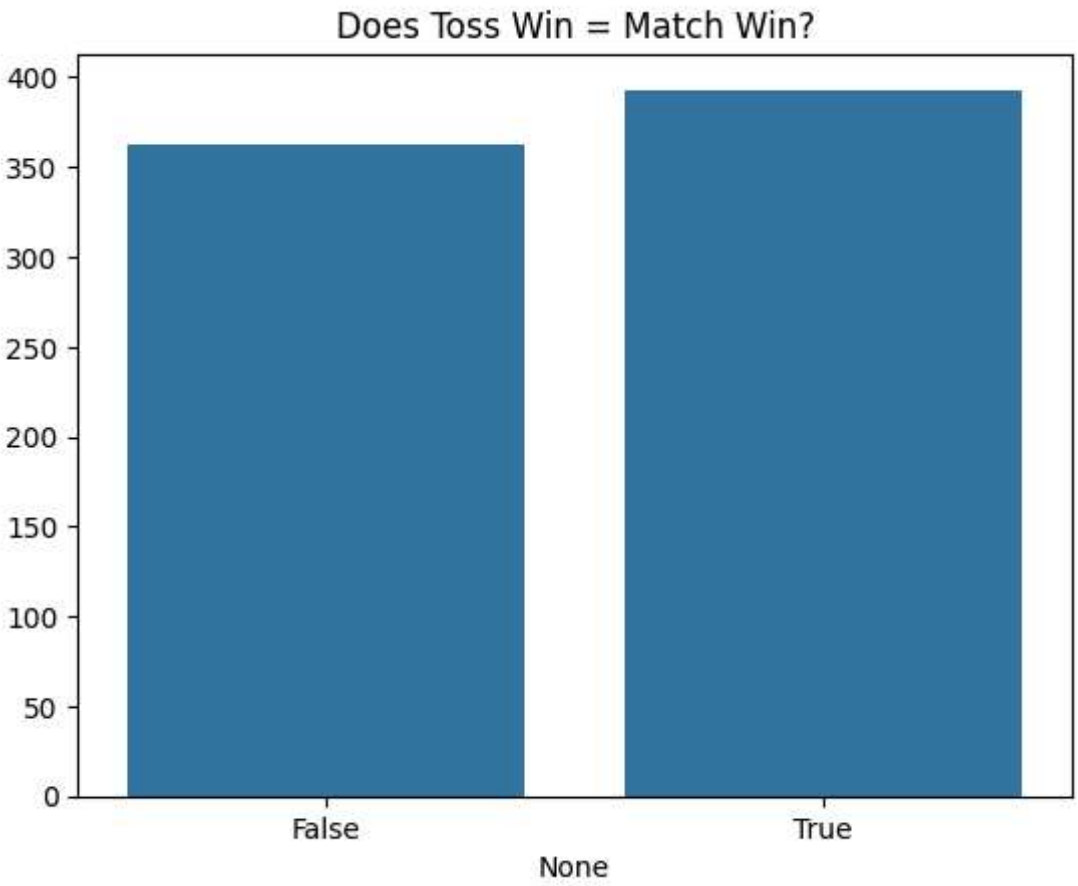
• Chris Gayle and AB de Villiers lead the chart, highlighting the massive impact of international explosive batsmen on match outcomes.

• The presence of MS Dhoni and RG Sharma shows that consistent leadership and finishing ability are key to winning "Man of the Match" awards.

• These players are crucial for franchise branding and high audience engagement.

In [34]:
```
# Toss decision vs Result [cite: 74, 75]
print("Toss decision counts:\n", m['toss_decision'].value_counts())
toss_win_match_win = (m['toss_winner'] == m['winner']).value_counts()
sns.barplot(x=toss_win_match_win.index, y=toss_win_match_win.values)
plt.title('Does Toss Win = Match Win?')
plt.show()
```

```
Toss decision counts:
 toss_decision
field    463
bat      293
Name: count, dtype: int64
```



Toss Decision vs. Result Insight:

• Captains chose to field first in 463 matches compared to batting first in only 293 matches.

• This suggests a strong strategic preference for chasing, possibly due to the dew factor or better target clarity.

• However, the "Does Toss Win = Match Win" graph shows that winning the toss only results in a match win about 52% of the time, proving that team performance is more important than luck.

Submitted by Shaswat Jaiswal BCADS 16 Univ. Roll No: 1250258419 Class Roll No: 26